

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра экономической математики, информатики и статистики (ЭМИС)

ШАБЛОНЫ

Отчёт по лабораторной работе по дисциплине «Объектно-ориентированное программирование»

Студент гр.594-2

А. А. Танцеров

“ 19 ” Октября 2025 г.

Руководитель канд. техн. наук, доцент
каф. ЭМИС ТУСУР

____ Я. В. Костелей

“ ” _____ 2025 г.

Томск 2025

Лабораторная работа №6

«Шаблоны»

Цель работы: изучение обобщённого программирования в C++ через разработку шаблонного класса для работы с информацией о приёме лекарственных средств.

Шаблоны классов представляют мощный механизм обобщённого программирования в языке C++, позволяющий создавать классы, которые могут работать с различными типами данных или конфигурациями без необходимости дублирования кода. Шаблон класса определяется с использованием ключевого слова `template`, за которым следуют параметры шаблона в угловых скобках. Параметрами могут быть типы данных (например, `typename T`) или целочисленные константы (например, `int SIZE`). При инстанцировании шаблона компилятор генерирует конкретную реализацию класса для указанных параметров. Это позволяет создавать универсальные и типобезопасные конструкции, такие как контейнеры (векторы, списки) или, как в данной работе, классы с настраиваемыми характеристиками.

Особенностью шаблонов является их обработка на этапе компиляции, что обеспечивает высокую производительность, поскольку исключаются накладные расходы времени выполнения, характерные для динамического полиморфизма. Шаботонные классы могут иметь параметры-константы, что позволяет задавать размеры массивов, значения порогов или другие фиксированные характеристики непосредственно в коде. Это особенно полезно, когда требуется создать несколько версий класса с разными конфигурациями, но одинаковой логикой поведения, как в случае с классом для учёта приёма лекарств, где количество времён суток может варьироваться.

Шаботонные классы могут участвовать в иерархиях наследования как в роли базовых, так и производных классов. Однако важно учитывать, что шаботонный класс, инстанцированный с разными параметрами, порождает разные, несвязанные между собой типы, даже если они имеют общий

шаблонный код. Для интеграции шаблонных классов в полиморфные структуры часто используют непрямой подход: создают общий абстрактный базовый класс (интерфейс), от которого наследуются конкретные инстанции шаблона, или применяют шаблоны с параметрами по умолчанию. В контексте данной работы шаблон `MedicineTemplate` может быть адаптирован для работы через единый интерфейс `IMedicine`, обеспечивая полиморфное поведение при хранении объектов в гетерогенных коллекциях. Это демонстрирует гибкость C++ в комбинировании различных парадигм программирования для создания сложных, но поддерживаемых систем. Задание. Оформите шаблон, где можно задавать константу для количества времени суток.

На первом этапе был разработан шаблонный класс `MedicineTemplate` с параметром `REQUIREDRECEPTION`, задающим количество времён суток для приёма лекарства. В шаблоне определены основные поля: динамическая строка для названия, день приёма и два статических массива фиксированного размера `REQUIREDRECEPTION` для хранения плана и факта приёма. Реализованы конструкторы (включая конструктор копирования с флагом), деструктор, селекторы и все необходимые методы, адаптированные для работы с шаблонным параметром размера массива. Особое внимание уделено обработке исключений при выходе индекса за границы массива.

На втором этапе шаблон был протестирован с различными значениями параметра: стандартный случай с 5 приёмами (соответствующий перечислению `TOD`) и альтернативные варианты (например, 3 приёма). Продемонстрирована работа всех методов, включая перегруженные операторы `<<` для вывода и `[]` для индексации, а также перегруженный метод `needToTake()` без параметров. Проверена корректность работы в гетерогенном массиве указателей на базовый интерфейс, подтверждающая полиморфное поведение шаблонного класса при правильной интеграции в иерархию наследования.

Шаблон класса представлен на листинге 1.

```

#pragma once
#include <string>
#include <iostream>
using namespace std;

template <int REQUIREDRECEPTION>
class MedicineTemplate {
private:
    char* name;
    unsigned int day;
    bool didIt[REQUIREDRECEPTION] = {0};
    bool planReception[REQUIREDRECEPTION] = {0};
    enum TOD { morning, before_dinner, dinner, supper, night, tod_size };

public:
    MedicineTemplate(const char* medicalName, int startDay) {
        if (startDay < 1 || startDay > 365) throw range_error("День вне
диапазона!");
        if (medicalName == nullptr || medicalName[0] == '\\0') throw
invalid_argument("Неверное название!");
        size_t len = strlen(medicalName);
        name = new char[len + 1];
        strcpy_s(name, len + 1, medicalName);
        day = startDay;
        for (int i = 0; i < REQUIREDRECEPTION; i++) didIt[i] =
planReception[i] = false;
    }

    MedicineTemplate(const MedicineTemplate& info, bool copy) {
        size_t len = strlen(info.name);
        name = new char[len + 1];
        strcpy_s(name, len + 1, info.name);
        day = info.day + 1;
        if (copy) {
            for (int i = 0; i < REQUIREDRECEPTION; i++) {
                planReception[i] = info.planReception[i];
                didIt[i] = false;
            }
        } else {
            for (int i = 0; i < REQUIREDRECEPTION; i++) planReception[i] =
didIt[i] = false;
        }
    }

    ~MedicineTemplate() { delete[] name; }

    friend ostream& operator<<(ostream& out, const MedicineTemplate&
ourObject) {
        out << "Лекарство: " << ourObject.getName() << " принимать в " <<
ourObject.getDay() << " день.";
        return out;
    }

    bool& operator[](TOD time) {
        int index = static_cast<int>(time);
        if (index < 0 || index >= REQUIREDRECEPTION) throw
out_of_range("Неверное время приема!");
        return planReception[index];
    }

    const char* const getName() const { return name; }
    int getDay() const { return day; }

    void print() const {

```

```

        cout << "Наименование лекарства: " << name << endl;
        cout << "Принял в " << day << " день." << endl;
        for (int i = 0; i < REQUIREDRECEPTION; i++) {
            cout << "Прием в " << i+1 << ": Надо - " << (planReception[i] ?
"Да; " : "Нет; ");
            cout << "сделал - " << (didIt[i] ? "Да; " : "Нет; ") << endl;
        }
    }

    bool checkPlan(TOD time) const {
        int index = static_cast<int>(time);
        if (index < 0 || index >= REQUIREDRECEPTION) throw
out_of_range("Индекс вне границ!");
        if ((!planReception[index] && didIt[index]) || (planReception[index]
&& !didIt[index])) return false;
        return true;
    }

    void setPlan(TOD time, bool shouldTake) {
        int index = static_cast<int>(time);
        if (index < 0 || index >= REQUIREDRECEPTION) throw
out_of_range("Неверное время приема!");
        planReception[index] = shouldTake;
    }

    void setDid(TOD time, bool taken) {
        int index = static_cast<int>(time);
        if (index < 0 || index >= REQUIREDRECEPTION) throw
out_of_range("Неверное время приема!");
        didIt[index] = taken;
    }

    bool needToTake(TOD time) const {
        int index = static_cast<int>(time);
        if (index < 0 || index >= REQUIREDRECEPTION) throw
out_of_range("Неверное время приема!");
        return planReception[index];
    }

    bool needToTake() {
        for (int i = 0; i < REQUIREDRECEPTION; i++) {
            if (planReception[i] != didIt[i]) return false;
        }
        return true;
    }

    bool isCorrect(TOD time) const {
        int index = static_cast<int>(time);
        if (index < 0 || index >= REQUIREDRECEPTION) throw
out_of_range("Неверное время приема!");
        return planReception[index] && didIt[index];
    }
};

```

Листинг 1 – Шаблон класса Medicine

Вывод: изучено обобщённое программирование в C++ через разработку шаблонного класса для работы с информацией о приёме лекарственных средств.